

DOCUMENTO DE PRUEBAS

Sistema Mercurio / Solaire's Customer Manager (SSCM)

Next.js 15 + Prisma + SQLite

Fecha: Mayo 2026

Elaborado por: Kevin

Contenido

1. Pruebas Unitarias (PU-01 a PU-35)
2. Pruebas de Integracion (PI-01 a PI-15)
3. Pruebas Funcionales (PF-01 a PF-20)
4. Pruebas No Funcionales (PNF-01 a PNF-15)
5. Pruebas de Regresion (PR-01 a PR-10)

1. Pruebas Unitarias

Verifican el correcto funcionamiento de cada funcion o validacion de forma individual. Se centran en las validaciones de campos, condiciones de error y logica aislada de cada endpoint y componente del frontend.

ID	Modulo	Caso de Prueba	Entrada	Resultado Esperado	Resultado Obtenido	Estado	Ruta enCodigo
PU-01	Login	Campos vacios rechazados	username="" password=""	Error: 'Campos requeridos' (400)	Error 400 mostrado	OK	app/api/auth/login/route.ts (linea 9: if (!username !password))
PU-02	Login	Usuario inexistente o inactivo	username='noexiste' password='123'	Error: 'Usuario no encontrado' (401)	Error 401 mostrado	OK	app/api/auth/login/route.ts (linea 12: if (!user !user.activo))
PU-03	Login	Contraseña incorrecta	username valido, password incorrecto	Error: 'Contraseña incorrecta' (401)	Error 401 mostrado	OK	app/api/auth/login/route.ts (linea 15: if (!valid))
PU-04	Login	Login exitoso establece cookie	Credenciales validas	Cookie httpOnly con maxAge=8h , redirige	Cookie establecida, respuesta ok:true	OK	app/api/auth/login/route.ts (linea 18-22: res.cookies.set)
PU-05	Login (Front)	Validacion campo usuario vacio	Click login sin usuario	Mensaje: 'El usuario es obligatorio'	Mensaje mostrado en pantalla	OK	app/login/page.tsx (linea 17: if (!username.trim()))
PU-06	Login (Front)	Validacion campo contraseña vacia	Click login sin contraseña	Mensaje: 'La contraseña es obligatoria'	Mensaje mostrado en pantalla	OK	app/login/page.tsx (linea 18: if (!password))
PU-07	Registro	Campos obligatorios vacios	nombre="" username="" password=""	Error: 'Todos los campos son requeridos' (400)	Error 400 mostrado	OK	app/api/auth/register/route.ts (linea 8: if (!nombre?.trim() ...))
PU-08	Registro	Contraseña menor a 6 caracteres	password='123'	Error: 'La contraseña debe tener al menos 6 caracteres' (400)	Error 400 mostrado	OK	app/api/auth/register/route.ts (linea 11: if (password.length < 6))
PU-09	Registro	Username duplicado	username ya existente	Error: 'El nombre de usuario ya esta en uso' (409)	Error 409 mostrado	OK	app/api/auth/register/route.ts (linea 14: if (exists))
PU-10	Registro (Front)	Contraseñas no coinciden	password != confirm	Error: 'Las contraseñas no coinciden'	Mensaje de error mostrado	OK	app/register/page.tsx (linea 17: if (form.password !== form.confirm))
PU-11	Productos	Nombre y precio obligatorios	nombre="" precio=null	Error: 'Nombre y precio valido son obligatorios' (400)	Error 400 mostrado	OK	app/api/base/productos/route.ts (linea 16: if (!nombre?.trim() precio == null precio <= 0))
PU-12	Productos (Front)	Validacion nombre vacio	Click crear con nombre=""	Mensaje: 'El nombre es obligatorio'	Mensaje mostrado	OK	app/dashboard/productos/page.tsx (linea handleCreate: if (!form.nombre.trim()))

PU-13	Productos (Front)	Validacion precio <= 0	precio='0' o vacio	Mensaje: 'El precio debe ser mayor a cero'	Mensaje mostrado	OK	app/dashboard/productos/page.tsx (linea handleCreate: if (!form.precio parseFloat(form.precio) <= 0))
PU-14	Ventas Base	Venta sin productos	items=[]	Error: 'La venta debe tener al menos un producto' (400)	Error 400 mostrado	OK	app/api/base/ventas/route.ts (linea 11: if (!items?.length))
PU-15	Ventas Base (Front)	Carrito vacio no permite enviar	Click registrar sin carrito	Mensaje: 'Agrega al menos un producto'	Mensaje mostrado, boton deshabilitado	OK	app/dashboard/ventas/page.tsx (linea handleSubmit: if (carrito.length === 0))
PU-16	Clientes SSCM	Nombre real y gamertag obligatorios	nombre_real="" nombre_juego=""	Error: 'Nombre real y nombre en juego son obligatorios' (400)	Error 400 mostrado	OK	app/api/sscm/clientes/route.ts (linea 8: if (!nombre_real?.trim() !nombre_juego?.trim()))
PU-17	Clientes SSCM	Gamertag duplicado	nombre_juego ya existente	Error: 'El nombre en juego ya existe' (409)	Error 409 mostrado	OK	app/api/sscm/clientes/route.ts (linea 13: catch => 409)
PU-18	Clientes SSCM	No eliminar cliente con ventas	Cliente con ventas > 0	Error: 'No se puede eliminar: tiene X ventas' (400)	Error 400 mostrado	OK	app/api/sscm/clientes/[id]/route.ts (linea 23: if (client._count.ventas > 0))
PU-19	Configuración	Campos numericos obligatorios	Valor NaN en algun campo	Error: 'Todos los campos son requeridos' (400)	Error 400 mostrado	OK	app/api/sscm/configuracion/route.ts (linea 7: if (vals.some(isNaN)))
PU-20	Configuración	Costo y precio > 0	costo_por_100v=0	Error: 'El costo y precio por 100V deben ser mayores a cero' (400)	Error 400 mostrado	OK	app/api/sscm/configuracion/route.ts (linea 8: if (vals[0] <= 0 vals[1] <= 0))
PU-21	Configuración	Cashback no negativo	cashback_regalo=-5	Error: 'No se pueden poner cantidades negativas' (400)	Error 400 mostrado	OK	app/api/sscm/configuracion/route.ts (linea 9: if (vals[2] < 0 vals[3] < 0))
PU-22	Ventas SSCM	Campos requeridos faltantes	type=null, total_vbucks=NaN	Error: 'Faltan campos requeridos' (400)	Error 400 mostrado	OK	app/api/sscm/ventas/route.ts (linea 22: if (!type isNaN(totalVbucks) isNaN(clientId)))
PU-23	Ventas SSCM	Cashback excede bolsa de pavos	vbucks_cashback > bolsa_pavos	Error: 'Solo tiene XV disponibles' (400)	Error 400 mostrado	OK	app/api/sscm/ventas/route.ts (linea 30: if (vbucksCashbackUsed > client.bolsa_pavos))
PU-24	Ventas SSCM	Cashback excede bolsa de pesos	pesos_cashback > bolsa_pesos	Error: 'Solo tiene \$X disponibles' (400)	Error 400 mostrado	OK	app/api/sscm/ventas/route.ts (linea 31: if (pesosCashbackUsed > client.bolsa_pesos))

PU-25	Ventas SSCM	Cashback no puede exceder total	Cashback > total_vbucks	Error: 'El cashback no puede exceder el total' (400)	Error 400 mostrado	OK	app/api/sscm/ventas/route.ts (linea 35: if (realVbucksPaid < 0))
PU-26	Ventas SSCM (Front)	Pavos minimo 200	total_vbucks=100	Error: 'El minimo son 200 pavos'	Mensaje mostrado	OK	app/sscm/ventas/page.tsx (totalVbucksError: if (v < 200))
PU-27	Ventas SSCM (Front)	Pavos multiplo de 100	total_vbucks=350	Error: 'Debe ser multiplo de 100'	Mensaje mostrado	OK	app/sscm/ventas/page.tsx (totalVbucksError: if (v % 100 !== 0))
PU-28	Usuarios	username, password y nombre requeridos	username=" password=" nombre="	Error: 'username, password y nombre son requeridos' (400)	Error 400 mostrado	OK	app/api/sscm/usuarios/route.ts (linea 13: if (!username !password !nombre))
PU-29	Usuarios	SSCM no puede ser admin	punto_de_venta='sscm' rol='admin'	Error: 'El punto de venta SSCM no puede tener rol de admin' (400)	Error 400 mostrado	OK	app/api/sscm/usuarios/route.ts (linea 16: if (pdv === 'sscm' && rolFinal === 'admin'))
PU-30	Usuarios	No deshabilitar admin	Desactivar cuenta admin	Error: 'No se puede deshabilitar una cuenta de admin' (400)	Error 400 mostrado	OK	app/api/sscm/usuarios/[id]/route.ts (linea 7: if (target?.rol === 'admin'))
PU-31	Periodos	Mes o anio invalido	month=13 year=abc	Error: 'Mes o anio invalido' (400)	Error 400 mostrado	OK	app/api/sscm/periodos/route.ts (linea 14: if (isNaN(m) m < 1 m > 12 isNaN(y)))
PU-32	Cuentas	Nombre obligatorio	nombre="	Error: 'El nombre es obligatorio' (400)	Error 400 mostrado	OK	app/api/sscm/cuentas/route.ts (linea 11: if (!nombre?.trim()))
PU-33	Recargas	Pavos deben ser > 0	pavos=0	Error: 'Los pavos deben ser mayores a cero' (400)	Error 400 mostrado	OK	app/api/sscm/recargas/route.ts (linea 11: if (!cuentald isNaN(pavos) pavos <= 0))
PU-34	Recargas	Costo no negativo	costo=-10	Error: 'No se permiten cantidades negativas en el costo' (400)	Error 400 mostrado	OK	app/api/sscm/recargas/route.ts (linea 13: if (isNaN(costo) costo < 0))
PU-35	Sesion	Sin sesion redirige o bloquea	Request sin cookie de sesion	Error: 'No autorizado' (401)	Error 401 mostrado	OK	app/api/base/productos/route.ts (linea 6: if (!session))

2. Pruebas de Integracion

Verifican la interaccion correcta entre multiples componentes del sistema: API con base de datos, frontend con backend, autenticacion con redireccion, y transacciones que involucran multiples tablas.

ID	Modulo	Caso de Prueba	Descripcion	Resultado Esperado	Resultado Obtenido	Estado	Rutas Involucradas
PI - 01	Auth + DB	Registro completo de usuario	Enviar POST /api/auth/register con datos validos, verificar creacion en BD y cookie	Usuario creado con hash bcrypt, cookie establecida, redirige a /dashboard	Funciona correctamente	OK	app/api/auth/register/route.ts -> prisma.usuario.create + bcrypt.hash + res.cookies.set
PI - 02	Auth + Redirect	Login redirige segun punto_de_venta	Login con usuario pdv=sscm vs pdv=base	pdv=sscm redirige a /sscm, pdv=base redirige a /dashboard	Redireccion correcta	OK	app/api/auth/login/route.ts (retorna punto_de_venta) -> app/login/page.tsx (router.push)
PI - 03	Productos + Sesion	CRUD requiere autenticacion	GET /api/base/productos sin cookie	Retorna 401 'No autorizado'	401 retornado	OK	app/api/base/productos/route.ts (getSession) -> lib/auth.ts
PI - 04	Venta + Stock	Venta descuenta stock de productos	Registrar venta con items, verificar stock post-venta	Stock decrementado por cantidad vendida via transaccion	Stock actualizado correctamente	OK	app/api/base/ventas/route.ts (tx.producto.update decrement) -> prisma.\$transaction
PI - 05	Venta SSCM + Cliente	Venta actualiza bolsa de cliente	Registrar venta tipo regalo, verificar bolsa_pesos del cliente	bolsa_pesos incrementada por cashback_generado	Bolsa actualizada	OK	app/api/sscm/ventas/route.ts (tx.cliente.update) -> bolsa_pavos / bolsa_pesos
PI - 06	Venta SSCM + Cuenta	Venta regalo descuenta pavos de cuenta	Registrar venta regalo con cuenta_id	Cuenta.pavos decrementado por pavos_total via transaccion	Pavos descontados	OK	app/api/sscm/ventas/route.ts (tx.cuenta.update decrement) -> linea cuenta check
PI - 07	Venta SSCM + Periodo	Venta crea periodo automatico si no existe	Registrar venta en mes sin periodo	Periodo creado automaticamente, desactiva anteriores	Periodo creado	OK	app/api/sscm/ventas/route.ts (linea 25-30: if (!period) -> prisma.periodo.create)
PI - 08	Delete Venta + Bolsa	Eliminar venta revierte cashback del cliente	Eliminar una venta existente	bolsa_pavos y bolsa_pesos revertidos a estado anterior	Bolsas revertidas	OK	app/api/sscm/ventas/[id]/route.ts (DELETE -> tx.cliente.update reversion)
PI - 09	Editar Venta + Bolsa	Editar venta recalcula bolsa del cliente	PUT a venta existente con nuevos valores	Bolsa revertida del estado anterior y actualizada con nuevos calculos	Recalculo correcto	OK	app/api/sscm/ventas/[id]/route.ts (PUT -> reversion + recalculo)
PI - 10	Reportes + Ventas	Reporte agrega datos de ventas y clientes	GET /api/sscm/reportes con period_id	Summary y clientSummary calculados correctamente de las ventas filtradas	Datos correctos	OK	app/api/sscm/reportes/route.ts -> prisma.ventaSscm.findMany + aggregation
PI - 11	Configuracion + Ventas	Ventas usan config vigente	Registrar venta sin configuracion previa	Error: 'No hay configuracion. Configurala primero' (400)	Error mostrado	OK	app/api/sscm/ventas/route.ts (linea 14: if (!config))
PI - 12	Sidebar + Rol	Sidebar filtra opciones por rol	Usuario operador vs admin	Operador no ve 'Usuarios', admin si	Filtrado correcto	OK	app/components/Sidebar.tsx (filter: item.href !== '/dashboard/usuarios' rol === 'admin')

PI - 13	Pagina Usuarios + Rol	Solo admin accede a /dashboard/usuarios	Operador intenta acceder	Redirige a /dashboard	Redireccion correcta	OK	app/dashboard/usuarios/page.tsx (if session.rol !== 'admin' redirect)
PI - 14	Logout + Cookie	Logout elimina cookie y redirige	POST /api/auth/logout	Cookie borrada (maxAge=0), frontend redirige a /login	Sesion cerrada	OK	app/api/auth/logout/route.ts (maxAge: 0) -> Sidebar.tsx (handleLogout -> router.push)
PI - 15	Pagina Root + Sesion	Pagina raiz redirige segun sesion	Acceder a / con sesion activa	Redirige a /sscm o /dashboard segun punto_de_venta	Redireccion correcta	OK	app/page.tsx (getSession -> redirect)

3. Pruebas Funcionales

Verifican los flujos completos de usuario desde la perspectiva del usuario final. Cada caso simula la interaccion real con la interfaz para validar que los requerimientos funcionales se cumplen de principio a fin.

ID	Modulo	Caso de Prueba	Pasos	Resultado Esperado	Resultado Obtenido	Estado	Paginas/Archivos
PF - 01	Login	Inicio de sesion completo	1) Ir a /login 2) Escribir usuario y contrasena 3) Click Ingresar	Se muestra el dashboard correspondiente al punto de venta del usuario	Dashboard cargado	OK	app/login/page.tsx -> app/api/auth/login/route.ts -> app/dashboard/page.tsx o app/sscm/page.tsx
PF - 02	Registro	Registro de nuevo usuario	1) Ir a /register 2) Llenar nombre, usuario, contrasena, confirmar 3) Click Crear cuenta	Cuenta creada y redirige a /dashboard automaticamente	Cuenta creada y redireccion exitosa	OK	app/register/page.tsx -> app/api/auth/register/route.ts
PF - 03	Productos	Agregar producto al catalogo	1) Ir a Productos 2) Click '+ Nuevo Producto' 3) Llenar nombre, precio, stock 4) Click Crear	Producto aparece en la lista con datos correctos	Producto listado	OK	app/dashboard/productos/page.tsx -> app/api/base/productos/route.ts (POST)
PF - 04	Productos	Editar producto existente	1) Click Editar en producto 2) Cambiar nombre/precio/stock 3) Click Guardar	Datos actualizados en la lista	Datos actualizados	OK	app/dashboard/productos/page.tsx (startEdit/handleEdit) -> app/api/base/productos/[id]/route.ts (PATCH)
PF - 05	Productos	Desactivar producto	1) Click Desactivar en producto 2) Confirmar en dialogo	Producto desaparece de la lista (soft delete)	Producto eliminado visualmente	OK	app/dashboard/productos/page.tsx (handleDelete) -> app/api/base/productos/[id]/route.ts (DELETE -> activo:false)
PF - 06	Ventas Base	Registrar venta completa	1) Ir a Nueva Venta 2) Click en productos del catalogo 3) Ajustar cantidades 4) Agregar notas 5) Click Registrar	Venta registrada, carrito vacio, stock actualizado, mensaje de exito	Venta completada	OK	app/dashboard/ventas/page.tsx -> app/api/base/ventas/route.ts (POST + transaccion)
PF - 07	Historial	Ver historial con detalles	1) Ir a Historial 2) Click en una venta para expandir	Se muestran items, cantidades, precios y total de la venta	Detalles visibles	OK	app/dashboard/historial/page.tsx -> app/api/base/ventas/route.ts (GET)
PF - 08	Cientes	Registrar nuevo cliente SSCM	1) Ir a Clientes 2) Click '+ Nuevo Cliente' 3) Llenar nombre real y gamertag 4) Click Registrar	Cliente aparece en lista con bolsa_pavos=0 y bolsa_pesos=0	Cliente registrado	OK	app/sscm/clientes/page.tsx -> app/api/sscm/clientes/route.ts (POST)
PF - 09	Cientes	Buscar cliente por nombre o gamertag	1) Escribir en campo de busqueda	Lista filtrada en tiempo real	Filtro funciona	OK	app/sscm/clientes/page.tsx (filtered = clients.filter)
PF - 10	Cientes	Ver perfil completo con historial	1) Click en un cliente de la lista	Se muestra nombre, gamertag, bolsas, y	Perfil completo visible	OK	app/sscm/clientes/[id]/page.tsx -> app/api/sscm/clientes/[id]/route.ts (GET + include ventas)

				historial de ventas			
PF - 11	Ventas SSCM	Registrar venta tipo regalo	1) Ir a Nueva Venta SSCM 2) Seleccionar cliente 3) Elegir 'Regalo' 4) Ingresar pavos, cuenta 5) Registrar	Venta registrada, bolsa_pesos actualizada con cashback, cuenta descontada	Venta completada	OK	app/sscm/ventas/page.tsx -> app/api/sscm/ventas/route.ts (POST tipo=regalo)
PF - 12	Ventas SSCM	Registrar venta tipo codigo	1) Seleccionar cliente 2) Elegir 'Codigo' 3) Ingresar pavos 4) Registrar	Venta registrada, bolsa_pavos actualizada con cashback en pavos	Venta completada	OK	app/sscm/ventas/page.tsx -> app/api/sscm/ventas/route.ts (POST tipo=codigo)
PF - 13	Ventas SSCM	Vista previa en tiempo real	1) Llenar datos de venta y observar panel derecho	Vista previa muestra pavos reales, cashback, ingreso, costo, ganancia	Preview actualizado	OK	app/sscm/ventas/page.tsx (calculatePreview, useEffect)
PF - 14	Ventas SSCM	Venta como sorteo (sin cobro)	1) Seleccionar 'Es sorteo' 2) Registrar	Venta sin ingreso ni cashback, costo calculado, ganancia \$0	Sorteo registrado	OK	app/sscm/ventas/page.tsx (es_sorteo) -> route.ts (if esSorteo: cashback=0, revenue=0)
PF - 15	Periodos	Crear y activar periodo	1) Click '+ Nuevo Periodo' 2) Seleccionar mes/año 3) Marcar como activo 4) Crear	Periodo creado, anteriores desactivados, ventas visibles al seleccionar	Periodo creado y activo	OK	app/sscm/periodos/page.tsx -> app/api/sscm/periodos/route.ts (POST)
PF - 16	Cuentas	Crear cuenta y registrar recarga	1) Crear cuenta 2) Click '+ Recargar' 3) Llenar pavos/costo 4) Guardar	Cuenta creada, pavos incrementados por recarga	Recarga aplicada	OK	app/sscm/cuentas/page.tsx -> app/api/sscm/cuentas/route.ts + app/api/sscm/recargas/route.ts
PF - 17	Configuración	Guardar nueva configuración	1) Ir a Configuración 2) Editar tasas 3) Click Guardar	Nueva config vigente, aparece en historial con etiqueta VIGENTE	Config guardada	OK	app/sscm/configuracion/page.tsx -> app/api/sscm/configuracion/route.ts (POST)
PF - 18	Reportes	Ver reporte por periodo	1) Ir a Reportes 2) Seleccionar periodo del dropdown	Resumen con totales y desglose por cliente actualizado	Reporte generado	OK	app/sscm/reportes/page.tsx -> app/api/sscm/reportes/route.ts (GET ?period_id)
PF - 19	Usuarios	Crear usuario con rol y punto de venta	1) Ir a Usuarios 2) Click '+ Nuevo Usuario' 3) Llenar campos 4) Registrar	Usuario creado con rol y pdv asignados, listado en tabla	Usuario creado	OK	app/dashboard/usuarios/UsuariosClient.tsx -> app/api/sscm/usuarios/route.ts (POST)
PF - 20	Usuarios	Activar/Desactivar usuario	1) Click Desactivar en un operador	Usuario marcado como inactivo (opacidad reducida, badge 'Inactivo')	Estado cambiado	OK	app/dashboard/usuarios/UsuariosClient.tsx (toggleActivo) -> app/api/sscm/usuarios/[id]/route.ts (PATCH)

4. Pruebas No Funcionales

Evalúan aspectos de calidad del sistema que no están directamente relacionados con funcionalidades específicas: seguridad, rendimiento, usabilidad, integridad de datos, mantenibilidad y consistencia visual.

ID	Tipo	Caso de Prueba	Criterio de Aceptación	Resultado Obtenido	Estado	Evidencia en Código
PNF-01	Seguridad	Contraseñas almacenadas con hash bcrypt	Las contraseñas nunca se almacenan en texto plano; se usa bcrypt con salt de 10 rondas	Hash bcrypt verificado en BD	OK	app/api/auth/register/route.ts (bcrypt.hash(password, 10)) y app/api/auth/login/route.ts (bcrypt.compare)
PNF-02	Seguridad	Cookies HttpOnly	La cookie de sesión tiene httpOnly=true para prevenir acceso desde JS del cliente	Cookie httpOnly establecida	OK	app/api/auth/login/route.ts (httpOnly: true, path: '/', maxAge: 60*60*8)
PNF-03	Seguridad	Sesión expira en 8 horas	maxAge de cookie = 28800 segundos (8 horas)	Cookie expira correctamente	OK	app/api/auth/login/route.ts (maxAge: 60 * 60 * 8)
PNF-04	Seguridad	Endpoints protegidos por sesión	Todas las rutas de API (excepto auth) verifican sesión activa con getSession()	401 retornado sin sesión	OK	app/api/base/productos/route.ts, ventas/route.ts, etc. (const session = await getSession(); if (!session))
PNF-05	Seguridad	Control de acceso por rol	Página de Usuarios solo accesible para rol 'admin'; operador es redirigido	Operador redirigido a /dashboard	OK	app/dashboard/usuarios/page.tsx (if session.rol !== 'admin' redirect('/dashboard'))
PNF-06	Seguridad	Username normalizado a minúsculas	Usernames se convierten a lowercase para evitar duplicados por mayúsculas	Normalización aplicada	OK	app/api/auth/login/route.ts y register/route.ts (body.username?.toLowerCase())
PNF-07	Integridad de datos	Transacciones atómicas en ventas	Ventas, recargas y operaciones críticas usan prisma.\$transaction para garantizar atomicidad	Datos consistentes ante errores parciales	OK	app/api/base/ventas/route.ts y app/api/sscm/ventas/route.ts (prisma.\$transaction)
PNF-08	Integridad de datos	Soft delete de productos	Los productos no se eliminan físicamente; se marcan como activo=false	Producto desactivado, historial preservado	OK	app/api/base/productos/[id]/route.ts (data: { activo: false })
PNF-09	Usabilidad	Feedback visual de estados	Botones deshabilitados durante carga, mensajes de éxito/error con colores diferenciados	UX fluida con indicadores claros	OK	Todas las páginas (disabled={saving}, opacity condicional, mensajes con color verde/rojo)
PNF-10	Usabilidad	Busqueda de clientes en tiempo real	El buscador de clientes filtra resultados mientras el usuario escribe sin necesidad de enviar	Filtrado instantáneo	OK	app/sscm/clientes/page.tsx y app/sscm/ventas/page.tsx (filteredClients, onChange)
PNF-11	Usabilidad	Navegación por teclado en dropdown	El selector de clientes soporta ArrowUp,	Navegación accesible	OK	app/sscm/ventas/page.tsx (onKeyDown handler con

			ArrowDown, Enter, Escape			ArrowDown, ArrowUp, Enter, Escape)
PNF-12	Rendimiento	Carga lazy de datos	Los datos se cargan bajo demanda; periodos cargan detalle solo al hacer click	Carga rapida sin datos innecesarios	OK	app/sscm/periodos/page.tsx (handleSelectPeriod: fetch solo al click)
PNF-13	Compatibilidad	Responsive del sidebar	El sidebar tiene width fijo de 220px y main tiene marginLeft de 220px	Layout consistente en pantallas >= 1024px	OK	app/components/Sidebar.tsx (width: 220) y layouts (marginLeft: 220)
PNF-14	Mantenibilidad	Separacion API / Frontend	Toda la logica de negocio esta en rutas API (app/api/); el frontend solo consume endpoints	Arquitectura limpia y desacoplada	OK	Estructura: app/api/* (backend) vs app/dashboard/*, app/sscm/* (frontend)
PNF-15	Consistencia visual	Dos temas diferenciados	Mercurio (Base) usa tema azul/purpura moderno; Solaire (SSCM) usa tema dorado/oscura elegante	Temas consistentes en cada modulo	OK	app/components/Sidebar.tsx (isSscm condicionales), colores definidos en cada pagina

5. Pruebas de Regresion

Verifican que las operaciones de modificacion (editar, eliminar, cambiar configuracion) no introduzcan efectos secundarios no deseados en datos existentes. Se ejecutan despues de cualquier cambio en el sistema.

ID	Escenario	Caso de Prueba	Accion Original	Verificacion de Regresion	Estado	Archivos a Verificar
PR-01	Editar cliente no afecta ventas	Cambiar nombre de cliente	PUT /api/sscm/clientes/[id] con nuevo nombre	Las ventas asociadas al cliente siguen mostrando los datos correctos en historial y reportes	OK	app/api/sscm/clientes/[id]/route.ts (PUT) -> verificar app/sscm/clientes/[id]/page.tsx y app/sscm/reportes/page.tsx
PR-02	Editar venta recalcula bolsas	Modificar pavos_total de una venta	PUT /api/sscm/ventas/[id] con nuevo monto	Bolsa_pavos y bolsa_pesos del cliente reflejan la diferencia entre el valor anterior y el nuevo	OK	app/api/sscm/ventas/[id]/route.ts (PUT -> reversion + recalcu bolsas)
PR-03	Eliminar venta revierte todo	Eliminar venta existente	DELETE /api/sscm/ventas/[id]	Bolsa de pavos y pesos revertidas; si habia cuenta asignada, sus pavos no se restauran (por diseno)	OK	app/api/sscm/ventas/[id]/route.ts (DELETE -> tx.cliente.update reversion)
PR-04	Nueva config no afecta ventas anteriores	Guardar nueva configuracion	POST /api/sscm/configuracion con nuevas tasas	Las ventas antiguas mantienen los calculos hechos con la config que tenian asignada (configuracion_id)	OK	app/api/sscm/ventas/[id]/route.ts (usa originalSale.configuracion, no la config actual)
PR-05	Cambiar periodo activo no pierde datos	Activar un periodo diferente	PUT /api/sscm/periodos/[id] para activar otro	Los datos del periodo anterior siguen disponibles al seleccionarlo; las ventas no se alteran	OK	app/api/sscm/periodos/[id]/route.ts (updateMany activo:false + update activo:true)
PR-06	Desactivar usuario no elimina datos	Desactivar usuario operador	PATCH /api/sscm/usuarios/[id] activo=false	El usuario no puede iniciar sesion pero sus ventas y productos siguen en el sistema	OK	app/api/sscm/usuarios/[id]/route.ts -> app/api/auth/login/route.ts (if !user.activo -> 401)
PR-07	Crear producto no altera existentes	Agregar nuevo producto	POST /api/base/productos	Los productos existentes mantienen nombre, precio y stock inalterados	OK	app/api/base/productos/route.ts (POST crea nuevo) -> GET lista todos con orderBy
PR-08	Venta base descuenta stock correctamente	Registrar multiples ventas seguidas	POST /api/base/ventas con mismos productos	Cada venta descuenta el stock exacto; no hay condiciones de carrera gracias a transaccion	OK	app/api/base/ventas/route.ts (prisma.\$transaction con decrement atomico)
PR-09	Recarga incrementa pavos sin afectar ventas	Recargar cuenta	POST /api/sscm/recargas	Pavos de cuenta incrementados; las ventas ya registradas no se modifican	OK	app/api/sscm/recargas/route.ts (tx.cuenta.update increment)

PR-10	Logout no corrompe datos de sesion	Cerrar sesion y volver a entrar	POST /api/auth/logout luego POST /api/auth/login	Datos del dashboard se cargan correctamente tras re-login; no hay cache de sesion anterior	OK	app/api/auth/logout/route.ts (cookie maxAge:0) -> app/api/auth/login/route.ts (nueva cookie)
-------	------------------------------------	---------------------------------	---	--	----	--

Resumen de Pruebas

Tipo de Prueba	Cantidad	Aprobadas
Pruebas Unitarias	35	35
Pruebas de Integracion	15	15
Pruebas Funcionales	20	20
Pruebas No Funcionales	15	15
Pruebas de Regresion	10	10
TOTAL	95	95